COIT11237

LECTURE NOTES

Authored by
Lachlan J. W. Wyer

Document Start Date: 11th of June 2026
Latest Update: June 17, 2026

Contents

1 Introduction to Databases **5**
- 1.1 What is MySQL?: . . . 5
- 1.2 Databases: . . . 5
- 1.3 Problems with Spreadsheets . . . 5
- 1.4 Database System Components: . . . 5
- 1.5 SQL: . . . 6
- 1.6 Personal DBs: . . . 6
- 1.7 Commercial DBs: . . . 6

2 SQL Part 1 **6**
- 2.1 Keys: . . . 6
  - 2.1.1 Primary . . . 6
  - 2.1.2 Composite . . . 6
  - 2.1.3 Foreign . . . 7
- 2.2 SELECT Clause: . . . 7
- 2.3 WHERE Clause: . . . 7
- 2.4 Ordering Rows: . . . 7
- 2.5 Having: . . . 7
- 2.6 Subqueries: . . . 8

3 SQL - Part 2 **8**
- 3.1 Joins of Tables: . . . 8
- 3.2 Union: . . . 8
- 3.3 Modifying Data: . . . 8

4 ER Diagrams **9**
- 4.1 Purpose: . . . 9
- 4.2 Database Definition: . . . 9
- 4.3 Entity Representation: . . . 9
- 4.4 Identifier: . . . 9
- 4.5 Relationships: . . . 9
- 4.6 Modelling ERDs: . . . 10
- 4.7 Weak Entity: . . . 10
  - 4.7.1 Id-dependent . . . 10
  - 4.7.2 Non-Identifying . . . 11

5 ER Digrams - 2 **11**
5.1 Associative Entities: . . . 11
5.2 Subtype: . . . 11
5.3 Notation: . . . 11
5.4 Recursive Relationships: . . . 12

6 Normalization **12**
6.1 Data Redundancy: . . . 12
6.2 Relational DBs: . . . 12
6.3 Relational Checklists: . . . 13
6.4 Candidate Keys: . . . 13
6.5 Primary Keys: . . . 13
6.6 Boyce-Codd Normal Form: . . . 13
6.6.1 Referential Integrity: . . . 13
6.6.2 NULL Values: . . . 13
6.6.3 Functional Dependancy: . . . 14
6.7 BCNF: . . . 14

7 Database Design **14**
7.1 ER Diagrams to Tables: . . . 14
7.2 Constraints: . . . 14
7.2.1 Checks . . . 14
7.3 Referential Integrity: . . . 15
7.4 Overall Steps: . . . 15

8 Database Administration - Part 1 **15**
8.1 DB Processing Environments: . . . 15
8.2 Transactions: . . . 16
8.3 ACID Transactions: . . . 16
8.4 Concurrency Control: . . . 16
8.4.1 Extreme Inaccessibility . . . 16
8.4.2 Extreme Accessibility . . . 16
8.4.3 The Lost Update Problem . . . 17
8.4.4 Dirty Reads . . . 17
8.4.5 Non-repeatable Reads . . . 17
8.4.6 Phantom Reads . . . 18
8.5 Data Read Problems Overview: . . . 18
8.6 Resource Locks: . . . 18
8.7 Two-Phase Locking: . . . 19
8.8 Deadlock: . . . 19
8.8.1 Optimistic . . . 19

8.8.2 Pessimistic . . . 19
8.9 Transaction Isolation Levels: . . . 20

9 Database Admin - Part 2 **21**
9.1 DB Security: . . . 21
9.2 DB Security Guidelines: . . . 21
9.3 DB Failures: . . . 21
9.4 Recovery via Reprocessing: . . . 22
9.5 Recovery via Rollback: . . . 22
9.5.1 Rollforward . . . 22
9.6 Database Administrator Responsibilities: . . . 22
9.7 Data Validation: . . . 23
9.7.1 "CHECK" Constraint . . . 23
9.8 MySQL Views: . . . 23
9.9 View Advantages: . . . 23
9.10 Stored Programs: . . . 24
9.10.1 Stored Procedures . . . 24
9.10.2 Triggers . . . 25

10 Business Integration Systems **25**
10.1 Transaction Systems: . . . 25
10.2 BI Systems: . . . 25
10.3 Data Mining: . . . 26
10.4 Integration of Datasets: . . . 26
10.5 ETL Data Transformation: . . . 26
10.6 Data Mart: . . . 26
10.7 DB Application in a Business: . . . 26
10.8 Distributed DB Processing: . . . 27
10.9 Big Data: . . . 27
10.10 Non-relational DBs: . . . 27
10.10.1 NoSQL DB: . . . 27

11 Cloud Systems **28**
11.1 What is Cloud Computing?: . . . 28
11.2 Cloud Service Models: . . . 28
11.2.1 Software as a Service (SaaS) . . . 28
11.2.2 Platform as a Service (PaaS) . . . 28
11.2.3 Infrastructure as a Service (IaaS) . . . 29
11.3 Cloud Computing: . . . 29
11.3.1 Advantages . . . 29
11.3.2 Disadvantages . . . 29

11.4 DB Cyber Attcks: 29

1 Introduction to Databases

1.1 What is MySQL?:

Structured Query Language (SQL) is the language used to manage relational database management systems (RDBMS). It follows the CRUD principles. In that it Create, Replace, Update, & and Deletes. SQL is used in both small & large scale applications.

1.2 Databases:

A database (DB) can be made up of multiple tables. The conventional naming is singular. For examples, a customers' information DB would just be named customer, not customers. The table also needs a primary key (PK) as a unique identifier. This is so a singular record can be found. Tables can also be linked together through foreign keys (FK) which is just a PK in another table. When this is done, a relational database is created.

1.3 Problems with Spreadsheets

1. Have redundant/wrong/out-of-date data present within the sheet. This can create conflicting copies throughout.
2. Multiple themes per row creates modification problems
    - Insertion anomalies
    - Deletion anomalies

1.4 Database System Components:

- Users use forms/reports to modify data
- Database application converts user inputs into SQL data, which is then sent to DBMS
- DBMS:
    - Processes the SQL
    - Enforces the DB rules
    - Provides concurrency, security, back-up and recovery
- Database is where the data is physically stored

1.5 SQL:

SQL is an international standard for the talking with DBs and their tables. For example:

```
SELECT *
FROM Table.name
WHERE ID = 1
```

1.6 Personal DBs:

These are typically just local DBs and tables stored on the system. They can be found with the localhost flags. A crucial part of local DBs is that they only support one connection at a time.

1.7 Commercial DBs:

Multiple connections with many things happening at once. Connections are usually facillitated through JAVA or C#.

2 SQL Part 1

2.1 Keys:

2.1.1 Primary

When creating a table in SQL it must have a unique identifier. For example:

EmployeeID	FirstName
1	Lachlan
2	Paul
3	Paul

2.1.2 Composite

Sometimes a comp PK is utilized to define N:M relational tables. As long as the pair of keys is unique, the table will not break. For example:

ProjectID	EmployeeID
1200	1
1300	1
1300	2
1400	2

2.1.3 Foreign

These keys allow relationships between tables to form. The link must be done back to a PK in the other table. When setting a FK during the creation of the table there are two points to understand:

1. Cascade Delete: Once the original PK is deleted all rows with it as a FK
2. Cascade Update: Same as delete, but with updating all FK rows

2.2 SELECT Clause:

Used to select rows of data from a table. There are only two compulsary parts to the statements. That being the column names and table name or names. To remove duplicate rows doing a SELECT DISTINCT statement will remove duplicate rows.

2.3 WHERE Clause:

Filters rows by constraining items that are in the column name. For example:
SELECT EmployeeName
FROM employee
WHERE EmployeeID = 1

Further constraints can be added to the WHERE clause. Such as, IN, NOT, ALL, BETWEEN, IS NULL/IS NOT NULL, and LIKE. The LIKE can be defined with % for infinite number of characters and _ for 1 character. The _ can be done multiple times (e.g. _ _ _, but joined together like ___) for a set defined number of characters.

2.4 Ordering Rows:

Use the ORDER BY after the WHERE. Ordering can either be defined DESC or ASC, with most DBMSs having ASC as the default.

2.5 Having:

This is basically just a WHERE clause for grouped functions. When in a statement with WHERE clauses, the WHERE clause is applied before the HAVING.

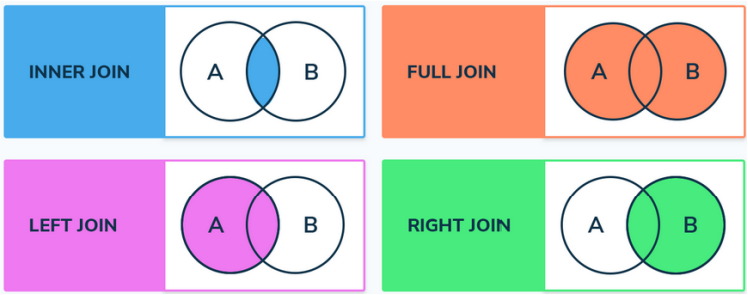
2.6 Subqueries:

SELECT *
FROM table1
WHERE column IN (subquery)

3 SQL - Part 2

3.1 Joins of Tables:

There a number of different types of joins. Joins can be used consecutaley to have multiple different tables joined together. The column used to join the table must be the same in each one. For example, an INNER JOIN is similary to,
FROM table1 as T1
INNER JOIN table2 as T2 ON T1.id = T2.id



3.2 Union:

These are used to completely join two tables together. This is similar to a full join. For example,
(SELECT * FROM table1)
UNION
(SELECT * FROM table2)

3.3 Modifying Data:

Used when updating/deleting a row of data. As well as creating or dropping a table.

4 ER Diagrams

A formal presentation of user requirements. They help refine the requirements before a database is created.

4.1 Purpose:

Help visualizes the relationships between the tables.

- What thing's are stored within the DB?
- What do we want to know about these things?
- Are there any relationships between the tables?
- Are there any constraints in the DB?

To summmarize ER diagrams help define the entire requirements of the DB system.

4.2 Database Definition:

A DB contains a definition of its data. Specifically, the meta-data is the specific data describing the structure of DBs data. These include the datatype (INT, DATE, CHAR & VARCHAR), uniqueness, auto-increment, and length.

4.3 Entity Representation:

Simply, name it an entity if it is more than just a name of something. I.E. Has non-key attributes or relationships with multiple entities.

4.4 Identifier:

Entity instances have an identifier. These are attributes that are unique (E.g. PKs) across all instances. Sometimes a composite of 2 or more identifiers to make a unique key.

4.5 Relationships:

Minimum cardinality: The minimum number of instances that must be involved with an instance.
Maximum cardinality: The maximum number of instances that a relationships can have.

Symbol	Meaning	Numeric Meaning
	Mandatory—One	Exactly one
	Mandatory—Many	One or more
	Optional—One	Zero or one
	Optional—Many	Zero or more

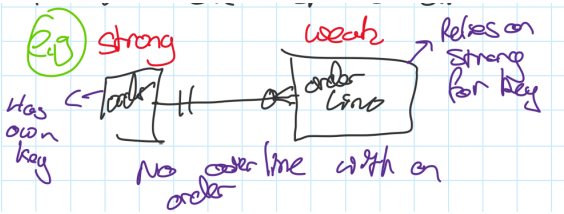
When drawing these, if connected by a dashed line, both entities are independent of each other.

4.6 Modelling ERDs:

1. Write out all entities
2. Display relationships and state any assumptions made with them
3. Check if any extra modelling is required

4.7 Weak Entity:

A WE is an entity that cannot exist in a DB without another entity also existing. That being if the strong entity is removed, the weak one must also be removed.



4.7.1 Id-dependent

This relies on its parent entity for its identifier. They usually have composite identifiers within the DB. Represented through a solid line.

4.7.2 Non-Identifying

Does not rely on its parent entity for its unique identification. Illustrated by a dashed line to represent the lack of dependency.

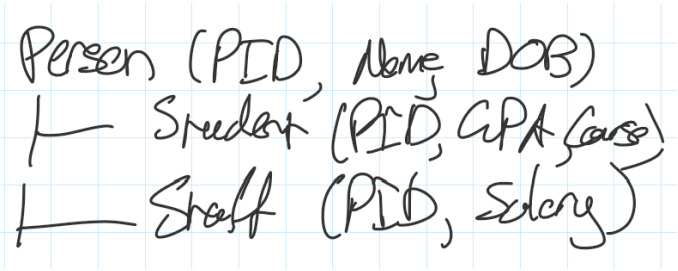
5 ER Digrams - 2

5.1 Associative Entities:

It is best practice to avoid N:M relationships between entities. To do this a third entity is made with 2 1:N relationships. This entity is then an id-dependent on both sides.

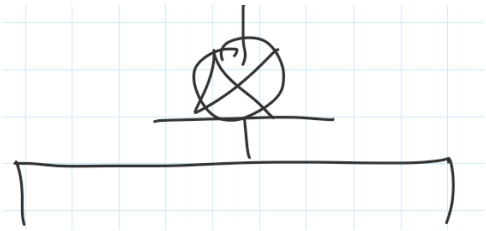
5.2 Subtype:

A type of entity that inherts the attributes and relationships of the more general super-type. Further specifc attributes can be added to the sub-type entities. Sub-type entities reduce reduncy by placing common atrributes in a super-type, whilst also keeping singular attributes in the sub-type.

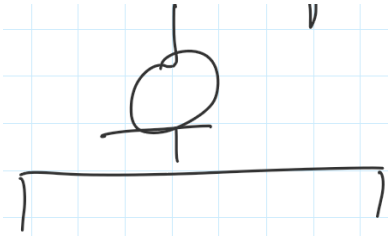


5.3 Notation:

Exclusive is related to atmost one sub-type.



Inclusive is where a super-type can be related to more than 1 sub-type.



Sub-type entities are not suitable when:

- Categories change frequently
- The difference is in a represented value not in its attributes
- Individuals move between categories over time

5.4 Recursive Relationships:

When a FK links too the PK within the same table. For example, an employee is supervised by a manager, who in turn an employee of the company.

6 Normalization

6.1 Data Redundancy:

List and spreadsheets store excess amount of redundant data. This can cause modification anomalies craeting conflicting copies/records in the table. They also have multiple themes per row. Where deletaion anomalies, delete everything within that record, and insertion anomalies, where blank cell are created when data is inserted.

6.2 Relational DBs:

Relational DBs split the spreadsheet into smaller tables that are linked through keys. They are more work to maintain up-front, but solves the problems associated with the spreadsheets.

6.3 Relational Checklists:

All following conditions must be met to make a spreadsheet a relational DB.

Characteristic	Met?
Rows contain data about just one entity	□
Columns contain data about attributes of the entity	□
Single value in each table cell	□
All entries in a column are of the same kind	□
Each column has a unique name	□
Order of columns is unimportant	□
Order of rows is unimportant	□
No duplicate rows	□

6.4 Candidate Keys:

Candidate keys are a unique set of keys that identify a single record. All these keys are capable of becoming a PK.

6.5 Primary Keys:

Notation is used to declare a relation. These include underlining PKs and italicized *FKs*. For exmaple,

TableName(PK, Col1, Col2, *Col3*)

6.6 Boyce-Codd Normal Form:

BCNF is based on a concept that an attribute determines another set of attributes. If nothing determines an attribute, a unique key is added to the system. This is known as a surrogate key.

6.6.1 Referential Integrity:

For tables to satisfy it, every FK value must match a PK value in another table.

6.6.2 NULL Values:

It is better to aviod having NULL values present within DB. This is because the specified value in the row and column is ambiguous.

6.6.3 Functional Dependancy:

Are when an attribute determines all other attribute. The main example for this is when a PK determines all values within a table.

6.7 BCNF:

The BCNF is a form of normalisation:

- Break up tables to remove anomalies
- Stream line data
- Stops replication of data points

7 Database Design

7.1 ER Diagrams to Tables:

Defining the data types present. This determines what will be placed within the column, INT or VARCHAR. Each column also needs to be assigned one specific data type. Further points can be NULL level status, whether a value can be a NULL value or whether a default value should be assigned. In addition the data in the column can also be constrained to a specifc type. E.g. a minimum birth year of 1800.
Each entity within an ERD should have its own table created.

7.2 Constraints:

Unique constraints is similar to a PK, in that each value must be unique to each other within that column. Columns that can have this are usernames or phone numbers. An advantage of this is that the table maintains an idex. Making the sorting of unique data faster.

7.2.1 Checks

Checks force values to be within a user defined range or values. If multiple check clauses are used, the standardised way it to have seperate check statements. A check can also reference multiple columns.

7.3 Referential Integrity:

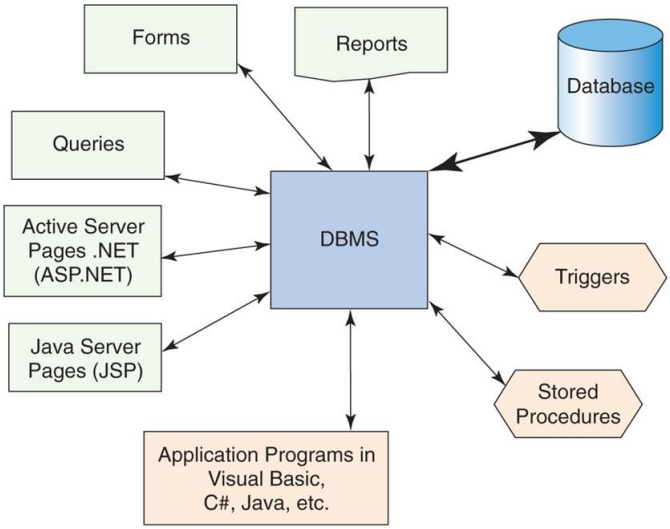
The FK needs to match PK in the parent table. When something is modified in the parent table, the FK needs to also be modified in the same way. For example, determining if when the FK in parent table is deleted do all rows with FK need to also be deleted.

7.4 Overall Steps:

1. Create a table for each entity
2. Identify the PK
3. Identify data types
4. Verify Normalisation
5. Identify id-dependent/id-independent weak entity relatioships
6. Create relatioships using FKs

8 Database Administration - Part 1

8.1 DB Processing Environments:



This is way of interacting with the DBMS from a user perspective. Convurrency is when multiple users are accessing and performing CRUD operations at once.

8.2 Transactions:

Some statements within SQL should be done in conjuction with another and never alone. If the entire query is not completed sucessfully it fails and the transaction is rolled back. For example:
Stock Transfer
UPDATE stock
SET Qty=Qty+5
WHERE Location = 'Warehouse'

8.3 ACID Transactions:

- Atomic - Everything in the transaction will COMMIT or nothing will
- Consistent - Database is consistent before and after the tractions
- Isolated - The actions of one transaction does not affect another
- Durable - All changes are perments (saved to non-volatile storage)

8.4 Concurrency Control:

Concurrency is needed in operations where multiple users are doing transactions on the same data. For this the accesibility scale is utilized.

8.4.1 Extreme Inaccessibility

- Data is inaccessable once a user updates
- Data updated is not available until the transaction commits
- Only one person can see data at a time

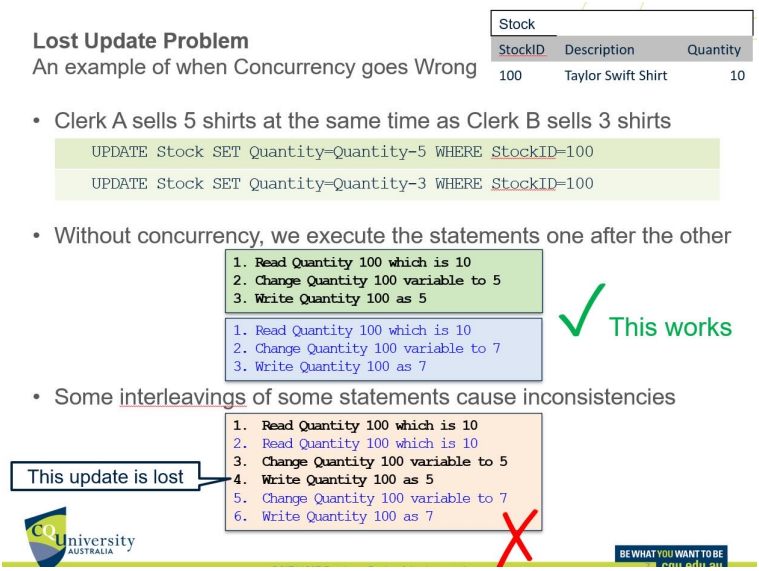
8.4.2 Extreme Accessibility

- Data is always radable
- Even when locked for updates

Balancing interdependcy (Changes required by one user impact another) and record retention (How long old data is accessible).

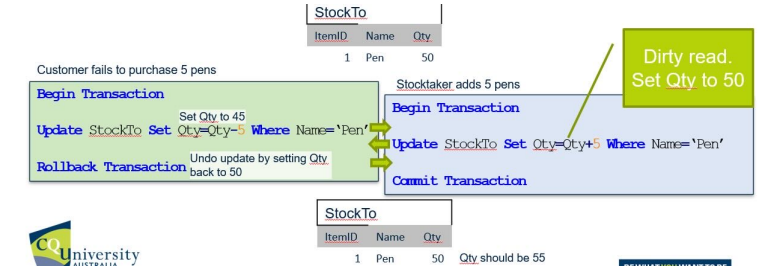
8.4.3 The Lost Update Problem

The update is overwritten by another transaction. As the old value is read and the new value is written over.



8.4.4 Dirty Reads

Dirty reads occur if the DBMS allow uncommitted updates as committed.



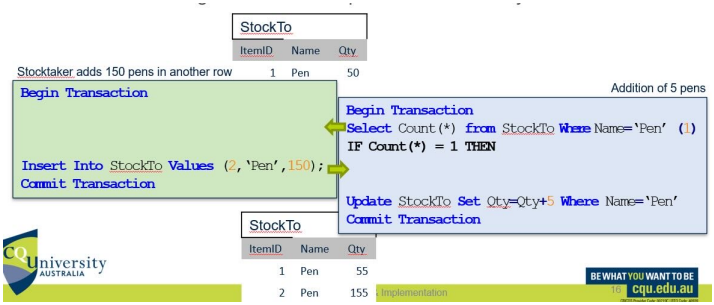
8.4.5 Non-repeatable Reads

Allow reading of committed changes during a repeating transaction.



8.4.6 Phantom Reads

See inserted reads partway through a transaction.



8.5 Data Read Problems Overview:

Data Read Problem Type	Definition
Dirty Read	The transaction reads a row that has been changed, but the change has *not* been committed. If the change is rolled back, the transaction has incorrect data.
Nonrepeatable Read	The transaction rereads data that has been changed, and finds changes due to committed transactions.
Phantom Read	The transaction rereads data and finds new rows inserted by a committed transaction.

8.6 Resource Locks:

Locks disallow reading and writing to a dataset whilst in place. A client must wait until a lock is released to perform the transaction. Explicit Locks are issued by the SQL developer for their specific application. Implicit locks are isssued by the DBMS and developers do not need to worry. For these types of locks to work effectively, everything must be synchronised to a serial order. This slows down transcation timing but everything is correct with not concurrency errors.

8.7 Two-Phase Locking:

Transactions are allowed to obtain locks but once the first lock is released, no new locks can be assigned. When obtaining locks this is known as the growing phase, and the releasing of locks is known as the shrinking phase. 2PL cannot prevent the phantom reads occuring.

8.8 Deadlock:

2PL transactions can suffer from something called deadlock. This occures when two transactions wait indefinitely for each other to release their lock. Almost every DBMS has algorithms in place to detect and prevent deadlock. When a deadlock occures, roll-back of one of the transactions to remove its changes occurs. Locking strategies can include:

- Optimistic
- Pessimistic

8.8.1 Optimistic

Based on the assumption that no conflict will occur.

1. Data is read
2. Data is processed
3. Updates issued
4. Check to see if a conflict occured
    - No conflict - Finish the transaction
    - Conflict - Repeat until processed without conflict (Locks can be placed)

8.8.2 Pessimistic

Uses the assumption that in all transactions a conflict will occur.

1. Locks issued
2. The transaction is processed
3. Locks released

Optimistic locking is generally faster as locks are held for less time. However transactions might need to be repeated numerous times to not have errors. Pessimistic locking is used when the risk of conflict is high or systems are always connected to the DBMS.

8.9 Transaction Isolation Levels:

0. Read Uncommitted
    - Lowest isolation level
    - Data inconsistency and integrity issues
    - Allows dirty reads, non-repeatable reads and phantom reads
1. Read Committed
    - Only read committed data
    - Only stops dirty reads
    - Allows non-repeatable reads and phantom reads
2. Repeatable Read
    - Locking rows that are read or modified until completion
    - Stops dirty reads and non-repeatable reads
    - Does not prevent phantom reads
3. Serialized
    - Highest isolation level
    - Prevents all concurrency issues
    - Slowest

		Isolation Level			
		Read Uncommitted	Read Committed	Repeatable Read	Serializable
Problem Type	Dirty Read	Possible	Not possible	Not possible	Not possible
	Nonrepeatable Read	Possible	Possible	Not possible	Not possible
	Phantom Read	Possible	Possible	Possible	Not possible

In theory there is no matching between 2PL and isolation levels.
Within MySQL, it defaults to level 2.

9 Database Admin - Part 2

9.1 DB Security:

- Authentication: Who are you?
- Authorization: Are you allowed to do that?

9.2 DB Security Guidelines:

- Run the DBMS behind firewalls.
- Keep everything updated with latest security patches.
- Limit DBMS functionality to smallest amount. This is the principle of least privilge.
- Protect the computer running the DBMS
  - Anti-virun software
  - Locks
  - Access
  - High Availability
- Manage Accounts and Passwords

9.3 DB Failures:

Common failure points include:

- Hardware failures
- Programming bugs
- Human error/mistakes
- Malicious actions

As they are very hard to prevent completely, recovery procedures are essential.

9.4 Recovery via Reprocessing:

1. Start from an earlier back-up
2. Reapply all transactions that happened after that back-up

Reprocessing is:

- Costly due to re-entering all data
- Risky due to increased amount of human-error, records of all transactions not being there, and some operations producing different results when re-run.

9.5 Recovery via Rollback:

Simply just an undo button. To undo a transaction, the log must contain a copy of the database before any change. This is known as a before image. A transaction can be undone by applying before images of changes to the DB.

9.5.1 Rollforward

Rollforward is the opposite of a rollback. An older state uses after images of the DB to re-apply transactions.

9.6 Database Administrator Responsibilities:

- Service system
  A system to gather, record and prioritise user reported problems.
- Change request process - change Management, create and manage processees for controlling DB config;
  - Procedure for recording change request
  - Conducting user and developer reviews of such request
  - Creating projects and tasks
- Documentation
  - Ensure documentationis maintained
  - Database structure
  - Concurrency control
  - Security

- Back-up and recovery
- Applications utilized

9.7 Data Validation:

Ensures data within a certain range of values. Keep data consistency within the definition. The main types that SQL uses is CHECK, UNIQUE, and NOT NULL.

9.7.1 "CHECK" Constraint

- Limitation are used at a DB Level
- Validates data
- Evaluated on INSERT, UPDATE, REPLACE and LOAD DATA elements

An error is raised if a constraint evaulation goes FALSE.

9.8 MySQL Views:

A view can be treated as a virtual table consisting of only columns and rows specified in a SELECT statement. For example:

```
CREATE VIEW view_name AS
    SELECT column_name
    FROM table_name
    WHERE column_name = "Hello World!";

SELECT * FROM view_name
```

A view is a subset of a database that can be saved. The view itself does not store any data, but reflects the most current data in base table. This is really good for complex queries.

9.9 View Advantages:

- Can limit the exposure of tables externals entities
- A view can restrict acess to seeing columns in the table
- It can simply queries by hiding complex retieval operations
- With restrictions views can update tables

9.10 Stored Programs:

There are 4 main types:

1. Stored procedure - Can be called from anything that has acess to the DB
2. Stored function - Called from SQL statement and returns a value
3. Trigger - Is executed in responce to an INSERT/UPDATE/DELETE statements on a specific table
4. Event - Is executed at a scheduled time

9.10.1 Stored Procedures

Calling a stored procedure without parameters

```
-- create a stored procedure in cqware
delimiter $$
 CREATE PROCEDURE SalesStoredProcedure()
 BEGIN
    SELECT SOID, PID, (NbrItemsRequested-NbrItemsDispatched) As QtyToBeSent
    FROM salesorderproduct;
 END
 $$
 Delimiter
 -- call the stored procedure
 Call SalesStoredProcedure();
```

A script calling parameters has some items to follow:

- IN parameters:
  - IN is the default mode
  - The program has to pass an argument to the procedure
- OUT parameters:
  - The value here can be charged inside the procedure and a new value passed back to the calling program

```
delimiter $$
CREATE PROCEDURE GetPartByCategory( IN categoryName VARCHAR(30))
BEGIN
SELECT *
FROM Product
Where Category = categoryName;
END
$$
delimiter ;

call GetPartByCategory('Glassware');
```

9.10.2 Triggers

A trigger is a special type of stored procedure that is used when a condition occurs BEFORE/AFTER/INSTEAD OF the transaction. Triggers provide a powerful way to enforce data consistency, log changes, ensure referential integrity and implement business rules.

10 Business Integration Systems

10.1 Transaction Systems:

These are processing systems that also support day-2-day routine transactions. Key features include:

- Support normal business activities
- Real-time
- Ensure all transactions are recorded

10.2 BI Systems:

Helps organizations use data to understand what has happened.

- Provide reports
- Support analysis of data
- Identification of patterns
- Predict future trends
- Control business direction

10.3 Data Mining:

This is the automated discovery of patterns with large datasets. These includes clustering, association detection, and statistical analysis.

10.4 Integration of Datasets:

There are two main options:

1. Connect seperate operational databases, which is often very costly
2. Data warehousing
    - Extract from operational DBs into a new DB
    - Reduces disruption from the day 2 day operations

10.5 ETL Data Transformation:

Used when operational DB needs to be cleaned before it goes into the warehouse. It is during the ETL process in which descions above data integrity problems are made.

10.6 Data Mart:

A collection of data that is smaller than a data warehouse. This is because it addresses a specific functional areas of the business.

10.7 DB Application in a Business:

OnLine Transaction Processing (OLTP):

- Process an order
- Calculate sales
- Operational systems
- Using a DB to run a business

OnLine Analytical Processing (OLAP):

- Highest selling products
- Report systems
- Using the DB to make good business descions

10.8 Distributed DB Processing:

Spreading the load and storage amongst several database severs.
Partioned - Breaking the DB into pieces
Replicated - Storing copies on multiple systems

10.9 Big Data:

DB involves large and complex datasets. These cannot be easily processed using traditional DB systems. These is no set definiton what Big Data is. However characteristics include; volume, velocity, variety, veracity.

10.10 Non-relational DBs:

Object oriented DBs have complex data structures and works well with OO programming languages.

10.10.1 NoSQL DB:

4 major types of NoSQL are:

- Key-value: Dynamo, Redis
- Document: CouchDB, MangoDB
- Column Family: Apache Cassandra
- Graph: Allegrograph, Titan

10.10.1.1 Advantages

- Scalability
- Simplicity
- Less code
- Easy maintainence
- Low latency
- Fast write speed

10.10.1.2 Disadvantages

- Less used
- Less consistent (No ACID)
- Less support (smaller technical community)
- No common language
- Difficult to understand data structures

11 Cloud Systems

11.1 What is Cloud Computing?:

It is computing technology that a user rents/leases a service/product over the internet from a service provider. Said service is usually provided by an IT giant. For example,

- Microsoft -> Azure
- Amazon -> AWS
- Alphabet -> Google Compute

The main features of cloud computing are:

- Remote hosting - Nothing is local
- Ubiquitous - Anywhere, Anytime
- Commodified - Pay as you go

11.2 Cloud Service Models:

11.2.1 Software as a Service (SaaS)

The SaaS model allows the use of applications as a service to end users. SaaS is a software delivery method that provides multi-tenant access to web-based functions. The bill based on usage, with a highly scaleable architecture.

11.2.2 Platform as a Service (PaaS)

PaaS provides the runtime environment for applications and development tools. Applications must be developed with a particular platform in mind.

11.2.3 Infrastructure as a Service (IaaS)

IaaS is the delivery of infrastructure as an on demand scalable service. IaaS provides access to fundamental compute resourse. E.g. HDD, SSD, CPU, GPU, and RAM.

11.3 Cloud Computing:

11.3.1 Advantages

- Lower local computer costs
- Improved performance
- Reduced initial costs and instant software access
- Scalability, easy upgrades

11.3.2 Disadvantages

- Requires constant, high-speed internet access
- Stored data will be lost once payments cease
- No digital soverignty

11.4 DB Cyber Attcks:

As more businesses move online, an increased demand for cybersecurity is required. The most attack vector is JS and SQL injections. To defend against specifically SQL injections:

- Sanatize all data inputs
- Validate inputs or give no choice
- Use parametrized queries
- Use properly stored procedures